Quantum Computer Simulations at Warp Speed: Assessing the Impact of GPU Acceleration

A Case Study with IBM Qiskit Aer, Nvidia Thrust & cuQuantum

Jennifer Faj, Ivy Peng, Jacob Wahlgren, Stefano Markidis
{faj,ivybopeng,jacobwah,markidis}@kth.se
Department of Computer Science
KTH Royal Institute of Technology, Sweden

2023 IEEE 19th International Conference on e-Science (e-Science) | 979-8-3503-2223-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/E-SCIENCE58273.2023.10254803

***Abstract*—Quantum computer simulators are crucial for the development of quantum computing. This work investigates GPU and multi-GPU systems' suitability and performance impact on a widely used simulation tool – the state vector simulator `Qiskit Aer`. In particular, we evaluate the performance of both Qiskit's default Nvidia `Thrust` backend and the recent Nvidia `cuQuantum` backend on Nvidia A100 GPUs. We provide a benchmark suite of representative quantum applications for characterization. For simulations with a large number of qubits, the two GPU backends can provide up to $14\times$ speedup over the CPU backend, with Nvidia cuQuantum providing a further $1.5-3\times$ speedup over the default Thrust backend. Our evaluation on a single GPU identifies the most important functions in Nvidia Thrust and cuQuantum for different quantum applications and their compute and memory bottlenecks. We also evaluate the gate fusion and cache-blocking optimizations on different quantum applications. Finally, we evaluate large-number qubit quantum applications on multi-GPU and identify data movement between host and GPU as the limiting factor for the performance.**

***Index Terms*—GPU, State Vector Quantum Computer Simulator, `Qiskit Aer`, Performance Characterization**

I. Introduction

Accelerators and GPUs critically impacted both HPC applications and machine learning workloads. Outstanding advancement examples are the acceleration of molecular dynamics software [1], molecular docking [2], computational fluid dynamics [3], plasma codes [4], and weather forecast [5]. GPUs revived deep-learning applications after the long AI winter [6]. In the current days, there is no efficient and large-scale deep neural network training without accelerators and specialized hardware. This study focuses on understanding whether GPU acceleration is a key technology for an emerging HPC application that is the usage of classical computing for simulating current and upcoming quantum computers. The question we want to answer in this work is: *are GPUs suitable and a key enabling technology for the acceleration of quantum computer simulations?*

Quantum computer simulators are a key tool for the development of quantum computing. In fact, the design and implementation of large-scale, reliable quantum computing infrastructure (hardware, software, and algorithm) require the deployment of quantum computer simulators. These tools can drive quantum computer design choices, prototype quantum algorithms in ideal and noisy controlled environments, and verify the correctness of the real quantum computer. Most of the current quantum algorithmic development depends on prototyping the algorithms and running them on quantum computer simulators to assess the correctness of the results in controlled environments and the impact of noise and error on the algorithms.

From a general standpoint, two main categories exist for quantum computer simulators. The first category models quantum computers closer to the hardware and describes how to control pulses (typically microwave pulses in superconducting and trapped ion systems) to implement quantum circuit operations. These simulators go under the name of pulse-level simulators. Examples of these are the IBM OpenPulse [7] and PASQAL Pulser [8]. The second approach is at a higher-level and much more common in use. It uses abstractions, such as quantum gates and circuits. This class includes several simulation techniques, among which state vector (also called *Schrödinger*), tensor network contraction [9], and Feynman path [10] techniques are the most important ones. Currently, the most established quantum simulator is the state vector quantum computer simulator because of its simplicity and ease of implementation. Different from other algorithms, the state vector is not suitable for noisy simulations (the density matrix technique is a convenient approach instead), and it is limited to a simulation with a relatively small number of qubits (the basic unit of information in quantum computing), e.g., ≤ 48 qubits on the current largest HPC systems [11]. On the other hand, tensor networks can simulate hundreds to thousands of qubits for low-entangled networks, but they do not provide the full state vector. The main limitation of state vector simulators is that memory usage scales exponentially with the number of qubits. For instance, a state vector simulation with 47 qubits requires the full memory of the Summit supercomputer, 2.8PB [11]. Because of these limitations, it is not clear whether the state vector quantum computer simulator might benefit from the usage of GPUs.

This work aims to evaluate the suitability of GPUs for state vector computer simulators and identify opportunities for their deployment on accelerated systems. To answer our research question, we use the IBM Qiskit Aer state vector simulator [12], a state-of-the-art quantum computer simulator providing two backends for Nvidia GPUs: one with Nvidia Thrust and

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

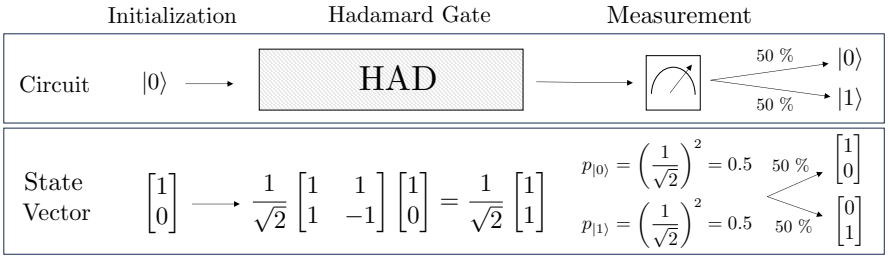


Fig. 1: A state vector simulator of a one qubit circuit with an Hadamard gate and a measurement. The Hadamard gate corresponds to a matrix-vector multiply. A measurement corresponds to a random sampling of a state according to its probability.

one with Nvidia cuQuantum.

The contributions of this paper are the following:

- We develop a benchmark suite of six representative quantum applications that support scaling the number of qubits in a state vector quantum computer simulator.
- We characterize and compare the performance of the state-of-the-art Qiskit Aer quantum computer simulator with three backends: CPU, GPU with Nvidia Thrust, and Nvidia cuQuantum on Nvidia A100 GPUs.
- We analyze the impact of two critical optimizations in transpilation – gate fusion and cache blocking, on different quantum applications.
- Our evaluation on a single GPU identifies the most important functions in Nvidia Thrust and cuQuantum for different quantum applications, and we built the roofline to identify compute and memory bottlenecks.
- Our evaluation on a multi-GPU setup identifies data movement between host and GPU as the top factor limiting the performance of large-number qubit quantum applications.

II. Quantum Computer State Vector Simulator

In this work, we focus on simulators using high-level abstractions, such as quantum gates and circuits, and the so-called state vector approach that stores all the information about quantum states.

Quantum Systems and Transformations with Complex Arrays and Matrices. The simplest quantum system in quantum computing is the quantum bit or *qubit*. This is a two-state quantum system that can be expressed with a linear superposition of its two orthonormal basis states (the $|0\rangle$ and $|1\rangle$ states) as $\psi = c_0\,|0\rangle + c_1\,|1\rangle$, where c_0 and c_1 are two complex numbers such that $\|c_0\|^2 + \|c_1\|^2 = 1$. In a state vector simulator, the qubit is represented by a two-element complex-value array as $[c_0\ c_1]^T$. An important abstraction is the quantum gate that allows us to manipulate the state of a quantum bit. Each quantum gate must be reversible and conserve the total probability of the states to one to obey the quantum mechanics law. Examples of single-qubit gates are the NOT (or X), rotation gates, and the most famous Hadamard gate to prepare an initial state into an equal superposition of states. In state vector quantum computer simulators, each one-qubit gate is expressed as a 2×2 unitary matrix (to enforce the reversibility and conservation of probability). For instance the Hadamard matrix is $\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$. For a comprehensive description of quantum gates and associated matrices, we refer to quantum computing textbooks [13]. A gate transformation on a qubit is expressed as a matrix-vector multiplication, where the matrix is a gate matrix, and the vector is the input state vector. If two or more gates are in sequence, it is possible to calculate the final effect of the gate sequence by simply multiplying the gate matrices. Because of this approach, the state vector calculations are dominated by small-size matrix-vector multiplications. In practice, quantum simulators do not store matrix information but are implemented in place (matrix-free). As noted in the seminal paper on HPC state vector simulators [14], the basic computational building block of the state vector simulator has relatively low arithmetic intensity (a matrix multiplication requires 14 FLOPs and 16B need to be moved when using single precision).

When we measure the qubit, its quantum state ψ collapses to either basis state of $|0\rangle$ or $|1\rangle$ with probabilities $p_0 = \|c_0\|^2$ and $p_1 = \|c_1\|^2 = 1 - \|c_0\|^2$. In a state vector simulator, the measurement outcome in a basis state is simply simulated with random sampling using the probability p_1 and p_2 to decide the measurement outcome. Multiple measurements or shots can be modeled with several sampling using the same full-state vector calculated by the simulator. Fig. 1 shows an example of a simple state vector quantum computer simulation of a one-qubit circuit with a Hadamard gate. The qubit is initialized in a classical basis state ($|0\rangle$ in the Fig. 1), corresponding to a state vector $[1\ 0]^T$. We multiply the Hadamard matrix by the input qubit to apply a Hadamard gate. As a result of this transformation, we obtain the state vector $\frac{1}{\sqrt{2}}[1\ 1]^T$: the qubit is an equal superposition of the basis states $|0\rangle$ and $|1\rangle$.

In quantum computing, multi-qubit systems are formed by using the tensor product of single-qubit systems. For instance, we can combine two single-qubit systems $q_1 = [c_0\ c_1]^T$ and $q_2 = [c_2\ c_3]^T$ and the result will be a multi-qubit system $q_{q_1 \otimes q_2} = q_1 \otimes q_2 = [c_0c_2\ c_0c_3\ c_1c_2\ c_1c_3]^T$. From this simple example of combining two qubits, it is clear that combining N qubit systems in one multi-qubit system will require 2^N complex elements in the state vector simulator, leading to an exponential increase of the memory requirements with N. For instance, a simulation with 31 qubits requires

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

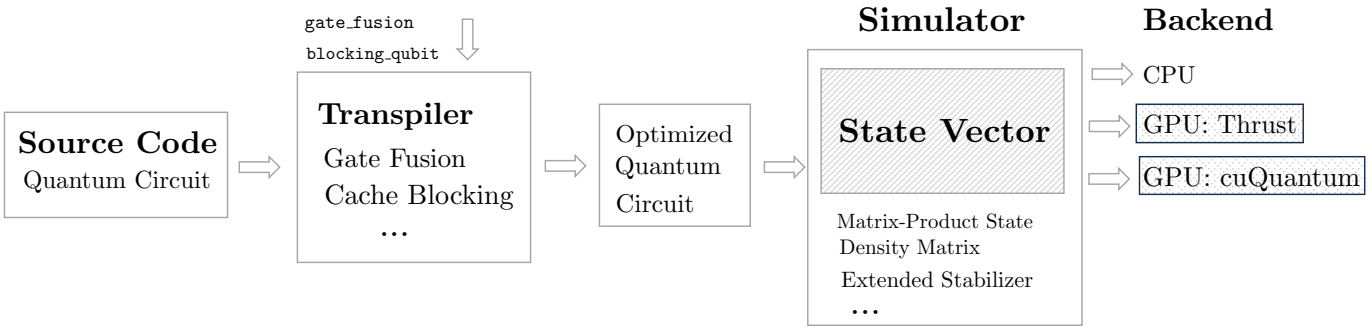


Fig. 2: The workflow of Qiskit Aer running state vector simulations.

$2^{31} \times 2 \times 4/10^9$GB $= 17.2$GB for the state vector array in single precision.

With multi-qubit systems, we can have one-qubit gate acting in parallel on different qubits: in this case the gate matrix acting on the full state vector can be calculated as the tensor product of the gate matrices. For instance, in the case of a two qubit system with two Hadamard gates, thanks to the tensor product, we can fuse two Hadamard matrices in one 4×4 matrix acting on the full state vector. In multi-qubit systems, we also have true multi-qubit gates, e.g. transformations that take as input the values of two or more qubits. The archetype of multi-qubit operations are the controlled operations, akin to classical conditional operations: a one-qubit gate/transformation is performed on a target qubit on the condition that a control qubit is in the state $|1\rangle$. An example of a multi-qubit gate is the two-qubit C-NOT and three-qubit version of it, called CC-NOT or Toffoli gate. Controlled gates introduce entanglement.

Qiskit Aer State Vector Simulator. Qiskit is an open-source software stack, initially released by IBM in 2017, for developing codes for quantum computers at the level of circuits, pulses, and algorithms. The Qiskit Aer component provides a range of high-performance quantum computing simulators, including the state vector simulator we use in this work, and a GPU port by Doi et al. [15], [16] using the Thrust library and recently also a cuQuantum [17] port. The open-source IBM Qiskit Aer programming framework [12] provides an ideal environment to evaluate state vector performance on GPU and compare it with the CPU version. Fig. 2 shows a high-level description of the workflow when running the IBM Qiskit Aer simulator.

When running a quantum computer simulator, the first step is to provide a description of the quantum circuit to be executed. In quantum computing, algorithms and codes are expressed as quantum circuits with multi-qubit and a combination of quantum gates and measurements. In Qiskit Aer, this can be formulated in Python code using quantum gates, libraries of quantum gates (for instance, the Quantum Volume and QFT circuits), or by loading circuit information from an input file in the QASM format [18], a standard description of the circuit in a style reminiscent of classical Assembly.

The second step is the Qiskit Aer transpilation phase. This is a critical part of any quantum computing software stack, and it is mainly responsible for circuit optimization and mapping to underlying quantum computer topology via several compiler passes. When using the state vector quantum computer simulator, the transpiler is responsible for performance optimizations that increase the computation intensity and minimize data movement. For our research, the two critical performance optimizing transpiler passes are:

- *Gate Fusion.* A key performance optimization technique in a state vector simulator is to fuse two or more individual gate matrices into a fused matrix. For instance, we can fuse gates acting on different qubits by taking the tensor product of the individual gate matrices acting on different qubits. This will increase the computational intensity of the matrix multiply. It is also possible to fuse two or more gates acting on a single qubit by taking the matrix multiplication (and respecting the order since matrix multiplication is not commutative). In Qiskit Aer, the parameter `fusion_threshold` determines the threshold that the number of qubits must be greater than or equal to enable fusion optimization. The default threshold is 14 qubits.
- *Cache Blocking.* On multi-GPU systems and, in general, on distributed memory systems, we need to divide the state vector on different *chunks* (to use Qiskit Aer terminology [16]), similarly to the domain decomposition in parallel computing. In addition to dividing the state vector, qubit reordering or remapping is used to decrease data exchanges between large numbers of qubit gates: these techniques require inserting swap gates in quantum circuits.

The result of the Qiskit transpiler step is a new quantum circuit that has been optimized for performance. The new quantum circuit is then simulated by the simulator. Qiskit Aer provides a number of simulator approaches and supported hardware. This work focuses on GPU backends for the state vector simulator. In particular, Qiskit Aer provides two backends to enable execution on Nvidia GPUs (including multi-GPU):

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

- *Nvidia Thrust.* The default GPU implementation in Qiskit uses Nvidia Thrust. Nvidia Thrust is a C++ template library that provides high-level parallel algorithms and data structures for programming GPUs [19]. It is part of the CUDA Toolkit, which is a set of tools and libraries provided by Nvidia for developing GPU-accelerated applications. However, Thrust allows developers to write code in C++ and leverage the power of GPUs without needing to write CUDA kernels or manage low-level GPU details directly.
- *Nvidia cuQuantum.* Recently, Nvidia provided the cuQuantum SDK for optimized libraries and tools for accelerating quantum computing simulations [17]. cuQuantum currently supports quantum circuit simulations based on state vector and tensor network methods and leverages Nvidia Tensor Core GPUs for speedup. The state vector cuQuantum library is a Qiskit Aer backend and can be easily executed. We note that cuQuantum is installed as a binary library.

III. Methodology

We develop the following suite of representative quantum applications and benchmarks to evaluate the GPU performance when running Qiskit Aer.

Quantum Volume (QV) Circuit [20] is an important benchmark circuit, composed of random instances of circuits, and used to measure the Quantum Volume metric (a measure of the largest possible quantum circuit that a quantum computer can execute reliably).

Quantum Fourier Transform (QFT) [21] is the quantum analog of the classical Fourier transform to convert a function in the time or spatial domain into its frequency domain representation. Probably the most important quantum algorithm, Shor's algorithm for the factorization into prime numbers, relies on QFT as a crucial step in factoring large numbers and solving the discrete logarithm problem efficiently on a quantum computer.

Quantum Random Circuit (QRC) Sampling [22] is a circuit proposed to demonstrate quantum computational supremacy: sampling from the output distribution of a large ($\approx> 50$ qubits) random quantum circuit is beyond the reach of classical computers, and for this reason, QRC serves as evidence for quantum computational supremacy. The QRC circuit consists of randomly chosen gates, including single-qubit gates and two-qubit gates, applied to a set of qubits. In this work, we use QASM circuits generated by the Google Cirq [23] circuit used in [22].

Grover's Circuit [24] is another famous important quantum algorithm used for searching unsorted data. The basic building block of Grover's circuit is the so-called Amplitude Amplification (AA) primitive that allows to convert a phase difference in quantum states into an amplitude difference (therefore directly measurable). The AA primitive consists of Hadamard and phase gates. The second building block is the oracle circuit that encodes the information about the target solution that we want to find.

TABLE I: Hardware and software setup.

Setup	Details
CPU (#)	AMD EPYC (1)
Cores	16
Clock frequency	3.0 GHz
Memory	128 GB DDR4
GPU (#)	Nvidia A100 (2)
Memory per GPU	40 GB HBM2
Theoretical peak memory BW per GPU	1448 GiB/s
Theoretical peak SP FLOPs per GPU	10.5 TFLOP/s
Compiler	GCC 8.5.0
MPI	OpenMPI 4.1.4
CUDA Toolkit	CUDA 11.5
Qiskit	0.12
cuQuantum	23.03.0

TABLE II: Benchmark setup for profiling.

#GPUs (precision)	Benchmark	# qubits	depth	# gates	non-local gates
1 (SP)	QV	31	10	150	100%
	QFT	31	62	511	94%
	RQC	31	12	332	18%
	Grover	30	11	67	3%
	GHZ	31	31	31	98%
	QW	16 (it=5)	285712	351083	44%
2 (DP)	QV	33	10	160	100%
	QFT	33	66	577	94%
	RQC	33	12	353	27%
	Grover	32	12	71	3%
	GHZ	34	34	34	97%

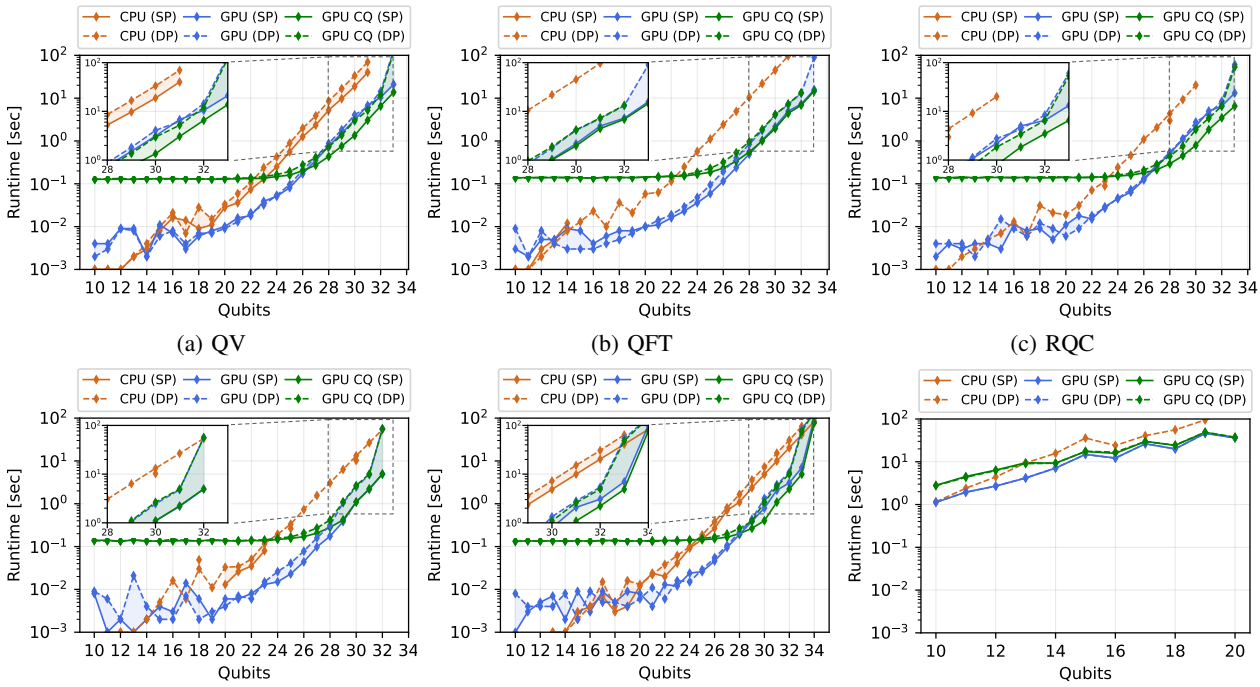
Greenberger-Horne-Zeilinger (GHZ) Circuit [25] is a circuit designed to create the so-called GHZ state that is a maximally entangled state: all qubits are in a superposition of being either all in the state $|0\rangle$ or all in the state $|1\rangle$.

1D Quantum Walk (QW) [26] is the quantum mechanics' version of classical 1D random walks, e.g., a walker is placed in a one-dimensional lattice and at each iteration a walker can shift either to the left or right lattice site with equal probability (in the quantum version this is implemented with a coin operator using a Hadamard gate). The beauty of quantum walks is that they allow the walker to exist in a superposition of states and paths, making quantum interference effects occur. Instead of randomly choosing a path, a quantum walker follows a set of quantum operators that determine its motion. QWs are used for searching algorithms (similar to Grover algorithms), quantum linear solvers, and quantum simulations. Different from the previously discussed quantum circuits, QW circuit depth depends on the number of iterations (or QW steps) and tends to be considerably large, even with relatively small numbers of qubits and iterations.

A. Experimental Setup

We use a multi-GPU system consisting of one AMD EPYC CPU and two Nvidia A100 GPUs for the experiments. Details on the specifications of the system and the software environment are provided in Table I. Since the focus of this work is the evaluation of GPU accelerators for state vector

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.



(a) QV (b) QFT (c) RQC

(d) Grover (e) GHZ (f) QW

Fig. 3: Qiskit Aer execution time for the CPU, GPU with Thrust (denoted GPU), and GPU with cuQuantum (denoted GPU CQ) backends when running the six applications using up to 34 qubits in single precision (SP) and double precision (DP). The usage of two GPUs allows to evaluate 31-34 qubits. The performance benefit of using GPUs is more than 10× for most of the application. For a large number of qubits, the cuQuantum backend for QV, RQC and GHZ is 1.5-3× faster than the Thrust GPU backend.

quantum computer simulators, we focus our profiling on the GPUs of the system, using Nvidia Nsight Systems [27] and Nsight Compute [28]. After performing scaling experiments, we utilize the Nvidia tools to obtain information regarding:

- the distribution of the runtime between GPU kernels and other processes.
- GPU kernels occupying the largest portion of the time.
- a roofline performance evaluation of each benchmark and the most used functions.
- memory footprints and data movement within each benchmark.

For each application, we chose a profiling setup that allows us to investigate *(i)* single precision performance on a single GPU and *(ii)* performance under full utilization of the system, i.e., using both GPUs and performing double precision computations. The characteristics of the chosen benchmark setups are shown in Table II. For the single GPU evaluation, the largest number of qubits that were still executable on a single GPU was chosen for each benchmark. Blocking and, thus, multi-GPU execution was enabled in the system, i.e., with any qubit number larger than the one stated in the upper half of Table II, two GPUs have been used automatically. Note that even the largest QW only uses a single GPU. The circuit depths for each benchmark are either fixed (QV, RQC) or depend on the number of qubits (all others). In Table II, we also present the total number of gates and the percentage of non-local gates (gates acting on one or more qubits, including controlled gates) after the circuit has been transpiled.

For all applications, five experiments in sequence (the same circuit runs five times in the same program) are carried out, and we show the average execution time over the experiments. The variance of the execution time is negligible; therefore, we omit the error bars when presenting the execution time.

IV. Evaluation

As a first step of our study, we study the execution time for CPU and GPU backends when increasing the number of qubits. Fig. 3 shows the scalability of the state vector method on all six benchmarks from 10 to 34 qubits when utilizing either the CPU (orange color) or two GPUs. For the latter, both Qiskit's default Thrust backend (blue) and Nvidia's cuQuantum backend (green) are evaluated. Note that some of the benchmarks could only be run in double precision on the CPU because single precision runs lead to segmentation faults. It can be observed that, with a low number of qubits, the runtime of each benchmark is lower when utilizing the CPU. At a break point between 14 to 18 qubits, the GPU with the Thrust backend becomes the fastest choice. The cuQuantum backend shows constant behaviour for up to $24 - 28$ qubits, and only afterwards becomes as fast as, or in some cases faster than the default Thrust backend of Qiskit. Specifically for the QV, RQC and GHZ benchmarks the cuQuantum backend is

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

approximately $1.5-3\times$ faster than the Thrust backend for $28-32$ qubits. By investigating the trace, the nearly constant execution time for the cuQuantum backend for a small number of qubits, i.e., $10-22$, is due to the large overhead of memory allocation and pinning that dominate the execution time.

Starting from 31 qubits in double precision, and 32 qubits in single precision, both GPUs on the system are utilized for both backends. Up to 34 qubits could be simulated in double precision on two GPUs for the GHZ benchmark. On the CPU the same benchmark could run with 34 qubits in single precision only. Here, the GPUs only reach a $1.1\times$ speedup and even a $0.8\times$ slowdown with cuQuantum and Thrust, respectively. For lower qubits a higher speedup is reached, e.g. for 30 qubits in single precision, the Thrust backend reaches a $6\times$ speedup and the cuQuantum backend a $12\times$ speedup. Similar behavior was found for the other benchmarks, where the GPUs generally reach a range between 32 and 33 qubits double precision, whereas the CPU could only simulate around 31 to 32 qubits for the same application, either not having sufficient memory available, or running for an exceedingly long time. With this maximum number of qubits the GPU outperforms the CPU with up to a factor of around $13-14\times$ for both the QV and QFT benchmarks in double precision with cuQuantum and Thrust.

As the QW application is characterized by a large depth, overall a lower number of qubits was executable in reasonable time spans. Furthermore, up to the maximum number of 20 qubits still only a single GPU is utilized. With the default Thrust backend the single GPU execution time is more than twice as fast as the CPU runtime after 13 and more qubits. With 20 qubits, the GPU with Thrust backend outperforms the CPU by a factor of nearly four. The cuQuantum backend is not beneficial for this benchmark. While it is close to the default Thrust backend with a larger number of qubits, there is still a difference of around 3-5 seconds between these two.

As we discussed in the background section, the gate fusion performance optimization could lead to a performance boost, especially when GPUs are used. By default, in Qiskit Aer quantum state simulators, the gate fusion is applied to circuits with a number of qubits greater than 14. We study the performance impact of using gate fusion and present it in Fig. 4. We observe a big impact of gate fusion for QV with Thrust backend, improving the performance in terms of execution time by a factor of $1.5\times$ (conversely, the transpiler gate fusion slows down the execution time of the cuQuantum backend). For the other applications, we do not observe a major impact of gate fusion.

After studying the scalability of quantum applications, we investigate GPU utilization and analyze the breakdown of the different activities. Fig. 5 presents compute activity (GPU kernels) in green, initialization in orange, finalization in yellow, and GPU idle time in brown. We can see that for the QV, QFT, and RQC benchmarks, the compute time dominates the execution time for both Thrust and cuQuantum backends, showing an efficient utilization of the GPU. The QW application is dominated by an initialization and GPU idle time

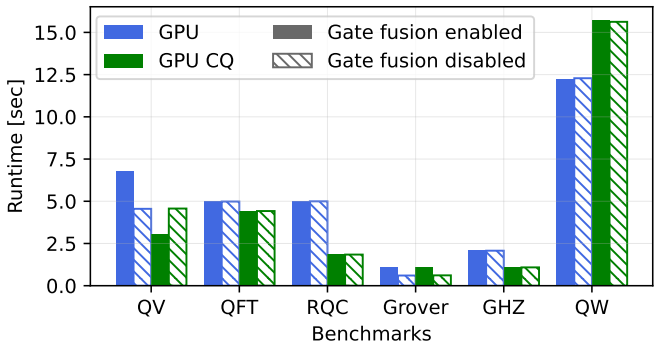
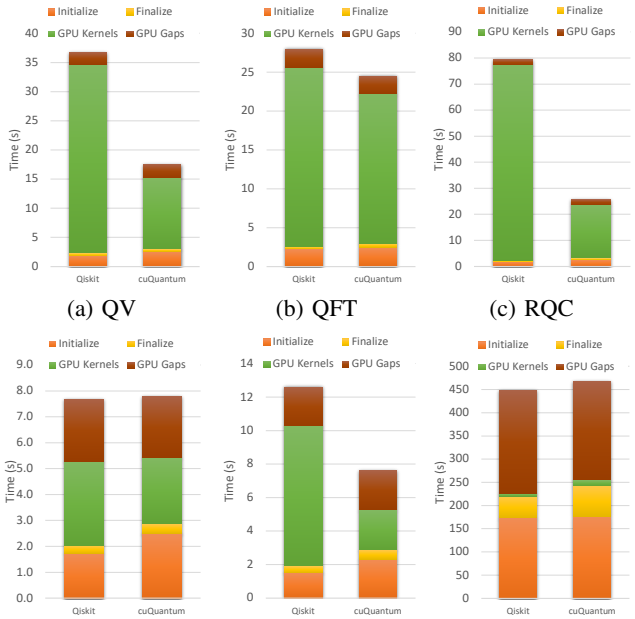


Fig. 4: Impact of using gate fusion for the different quantum applications.



(a) QV (b) QFT (c) RQC

(d) Grover (e) GHZ (f) QW

Fig. 5: Breakdown of the different GPU activities on one GPU. The GPU running kernels (compute) are represented in green color. GPU Gaps refers to the periods with no GPU activities. Data transfer from/to host and device memories are negligible. The QW application is dominated by the initialization that requires to move the circuit configuration to the GPU (QW has a very deep quantum circuit).

without efficient usage of the GPU. Most interestingly, we see cuQuantum can accelerate GPU kernels (green) by $1.2-3.7\times$ compared to the Thrust backend in all applications except the QW benchmark.

To further understand how the quantum applications use the GPU compute units and memory system, we identify with Nvidia Nsight Systems the GPU kernels that take the most time when running the two GPU backends and present them at the top of Fig. 6. We notice that matrix multiply is the dominant compute kernel when using the Thrust backend. Qiskit's Thrust backend provides a custom implementation for

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

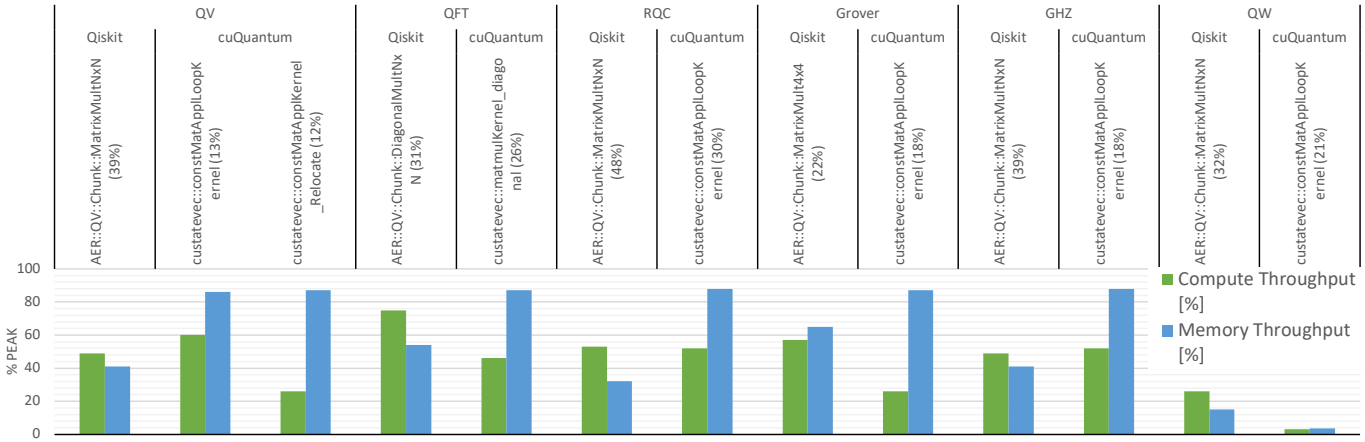


Fig. 6: Most time consuming (≥5%) GPU compute kernels (shortened function names) when simulating the largest number of qubits on a single GPU in single precision for each application. The compute and memory throughput, defined as the utilization of the theoretical peak throughput, respectively, in the top kernels used in the Thrust and cuQuantum backends. The cuQuantum GPU backend stresses memory throughput in all top functions more than Qiskit's Thrust backend (blue bar higher than green bar in all top functions).

multiplying with matrix sizes 2×2, 4×4 and 8×8 and matrix multiply with a generic $N \times N$ size. These large-size matrix multiplies are the result of the gate fusion optimization.

We present the percentage of compute and memory throughput in these most used kernels in Fig. 6. The compute throughput is calculated as the percentage of ALU pipe in active cycles and memory throughput is calculated as the percentage of DRAM in active cycles. The cuQuantum backend shows considerably higher memory and compute throughput than the Thrust backend, except for the Grover and QW applications. We also notice that, compared to Thrust, the cuQuantum backend always tends to stress memory more than compute, i.e., all blue bars are higher than green bars.

As last step in our performance characterization of the Qiskit GPU backends for state vector simulation, we present the roofline for the different kernels to determine whether the kernels are memory or compute bound in Fig. 7. Using the most utilized functions, we show the characteristics of each benchmark setup (Table II, 1 GPU (SP)) in a roofline model of one A100 GPU, with the ridge point at an arithmetic intensity of 6.8, 1448 GiB/s peak memory bandwidth and 10.5 TFLOP/s single-precision peak performance. The characteristics are obtained for the default backend (blue) and the cuQuantum backend (green). In Fig. 7, it can be seen that all applications fall in the memory-bound region except for the QW application, which has low bandwidth and compute intensity. However, the performance reached with the latter still states a big gap towards the roofline. When comparing the default backend with the cuQuantum backend, cuQuantum utilizes the compute resources more efficiently and, thus, reaches the roofline for all the other benchmarks, explaining the performance improvement observed earlier in Fig. 3.

For state vector quantum circuit simulations, the memory

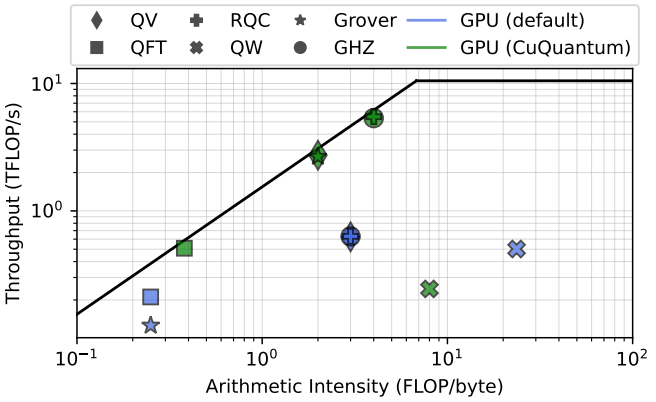


Fig. 7: Single precision roofline of the A100 GPU and performance of the most used kernel of the six applications. The cuQuantum GPU backend reaches the roofline for all the quantum applications except for QW.

usage for the state vector scales up exponentially with the number of qubits. Thus, a single GPU typically can only support simulations with qubits fewer than 32, as shown in the previous experiments. We further scale up the simulations by exploiting additional GPU memory in a multi-GPU setup.

When performing the simulation on multiple GPUs, the state vector is divided into different chunks, and communication is needed between the memory of the two GPUs. The most important factor determining the data movement is the number of Qiskit `blocking_qubits`. To showcase the impact of the `blocking_qubits` on two GPUs, we perform experiments with the QV benchmark with 33 qubits and both GPU backends in single and double precision, varying the number of blocking qubits. The results shown in Fig. 8 suggest that, while for single precision, the difference between 26-29 is

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

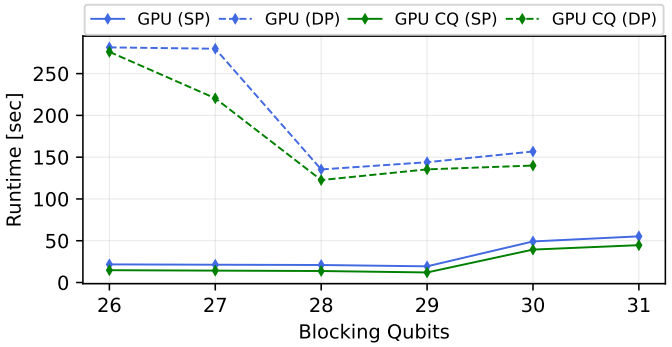


Fig. 8: Runtime with a varying number of `blocking_qubits` for a fixed number of 33 total qubits on 2 GPUs.

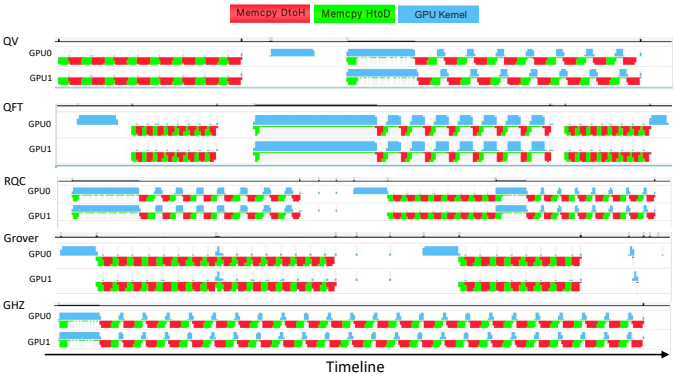
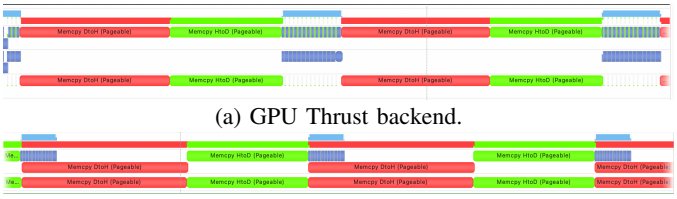


Fig. 9: Trace of the data movement and computation for all the applications on two GPUs, except for QW. The typical pattern is a large data movement from host to device (H2D) and device-to-host (D2H) before and after GPU kernel execution with little to no overlapping.

negligible, for double precision, a blocking factor of 28 is most beneficial for both backends. The same was confirmed for other benchmarks; thus, a factor of 28 was chosen for all other experiments.

When studying the performance on multi-GPU systems, it is critical to understand the memory footprint and data movement across the two GPU memories, as now the state vector is divided into different chunks residing on different memories. Investigating the memory footprint on two GPUs, the total memory usage is 72 GB for all the applications and the two GPU backends.

Overall, data movement between the host and GPUs becomes the top limiting factor of performance, taking more than 90% GPU time in all two-GPU experiments. We inspect the tracing of the quantum applications to understand the data movement. In Fig. 9, we show the trace of five applications using two GPUs with the Thrust backend. The traces show one experiment running the quantum circuit for the applications with the largest possible number of qubits (see II), except for QW, as it was only executed on a single GPU. Three colors represent three main phases: memory copy from the host to



(a) GPU Thrust backend.

(b) GPU cuQuantum backend.

Fig. 10: Comparison of memory transfer in the Thrust and cuQuantum backend multi-GPU strategy: cuQuantum overlaps kernel execution (blue) with `cudaMemcpyAsync` D2H data transfer (red). However, since the kernels are much shorter than data transfers, no large performance improvement in end-to-end simulation time is observed.

TABLE III: Runtime of the QV benchmark when utilizing two GPUs with the default Thrust backend, with and without MPI communication.

Qubits	Precision	Runtime [s] no MPI	Runtime [s] MPI
32	SP	10.456	15.194
	DP	14.349	108.060
33	SP	21.041	149.562
	DP	135.318	377.686

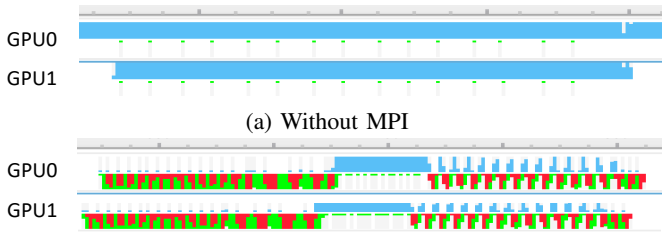
the device (green), compute (light blue), and memory copy from the device to the host (red). By analyzing Fig. 9, we see that data movement is similar for all the applications – there is an extensive data movement from host to device (H2D) and device-to-host (D2H) before and after GPU kernel execution with little to no overlapping. Note that Fig. 9 presents only a part of the whole timeline due to space limit.

In general, the traces obtained running the cuQuantum backend are similar to the ones with the Thrust backend. However, a main difference can be seen when comparing the traces. In Fig. 10, we show a zoom-in of the traces for the GHZ application comparing the Thrust and cuQuantum backends. We can see that, for the data transfer, the cuQuantum backend uses `cudaMemcpyAsync` to overlap kernel execution (light blue) with device-to-host data transfer (red). However, no overall performance improvement is observed because the kernels are much shorter than data transfers.

As we are using a multi-GPU node, we can use MPI to divide the workload between two GPUs. In an evaluation of distributed processing using MPI, we observe that the runtime increases significantly earlier when using MPI compared to the previous experiments that use an OpenMP-based distribution across the two GPUs of our single node setup (Table III).

Fig. 11 compares the traces with 33 qubits in single precision where the execution time deviates significantly. It can be observed that the high H2D and D2H transfers are already present in the MPI version of this setup, while the non-MPI version reaches this stage with 33 qubits in double precision, as shown earlier in Fig. 9.

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.



(a) Without MPI

(b) With MPI

Fig. 11: Comparison of traces with and without MPI on the Quantum Volume benchmark with 33 qubits in single precision on 2 GPUs.

V. Related Work

State vector quantum computer simulators have a long history of development. The first parallel HPC quantum computer simulation approaches have been proposed by Da Raedt et al [29], enabling the simulation of up to 36 qubits with the state vector representation, running benchmarks within 1,707 seconds when utilizing up to 4096 CPUs on the IBM BlueGene/L machine. Later on, Da Raedt et al [30] revised the massively parallel state vector simulator and provided additional performance measurements of up to 48 qubits on the Sunway TaihuLight and K supercomputers.

Now, supercomputers have evolved to include different types of accelerators, such as GPUs, whose utilization has been facilitated through high-level programming models, leading to the development of frameworks such as Qiskit that allow GPU-based quantum computer simulations. In this context, Doi et al. [15] perform scaling experiments on a heterogeneous CPU-GPU setup, showing the applicability of such a setup for quantum computer simulations. Their simulator has been integrated into the Qiskit Aer framework. In recent work, Doi et al. [16] implemented the cache-blocking technique used within Qiskit for multi-GPU acceleration. Using an IBM Power System AC922 with 6 GPUs, they simulate up to 35 qubits on a single node using the QV and QFT benchmarks, and perform further scaling experiments over multiple nodes. In addition, optimizations for multi-shot quantum simulations on GPUs have been carried out [31]. So far, our presented work shows an in-depth analysis of the performance of these approaches on a single node, leaving multi-node performance evaluation with a focus on MPI communication as a potential future work.

Imamura et al. [32] compare the aforementioned GPU performance to their CPU framework, mpiQulacs, focusing on distributed multi-node acceleration of the state vector simulator with MPI. They compare their accelerator's runtime (double-precision) with Qiskit Aer on the Quantum Volume benchmark with the same circuit depth used in our setup running on a GPU cluster with 6 Nvidia V100 GPUs per node. In their setup, mpiQulacs outperforms Qiskit after scaling to four nodes with 32 qubits. However, for the same benchmark on our setup, it was possible to scale well with up to 32 qubits on a single node with only two GPUs, requiring only 12 seconds, whereas the runtime on four CPU nodes with mpiQulacs lies above 20 seconds according to their results. This might be explained by the fact that there is no communication overhead in our setup since our evaluation is done on a single node only, which on the contrary, speaks for a higher efficiency of the GPU setup over the CPU setup.

VI. Discussion & Conclusion

In this paper, we assessed the potential of using GPUs to simulate quantum computers using the state vector approach that simulates the evolution of the quantum state complex array after applying a number of gate transformations. We evaluated the performance of different quantum applications with the IBM Qiskit Aer simulator that provides two backends for GPUs: one based on the Nvidia Thrust library and one based on the cuQuantum SDK.

We are now in the position of answering the initial research question: *are GPUs suitable and a key enabling technology for the acceleration of quantum computer simulations?* Brief answer: yes, they are. Overall, we found that GPUs can provide a large improvement in performance, with cuQuantum providing a major computational boost both in memory and compute throughput. In particular, for simulations with a large number of qubits, GPUs can provide up to $14\times$ performance improvements, with Nvidia cuQuantum being $1.5-3\times$ faster than the original Thrust backend for several of the presented benchmarks and reaching the roofline in our experiments. While the usage of cuQuantum's compute capability is outstanding, we note that Nvidia tensor core units [33] are not used. Their usage could provide an extra performance gain at the cost of reduced accuracy of the calculations (tensor cores work in mixed precision).

The major obstacle of state-vector simulations is that they quickly hit the memory and computational wall due to an exponential growth of computational and memory requirements with increasing numbers of qubits. One possibility is to use multiple computational nodes or disaggregated systems and memory pooling [34] for additional memory. However, we have shown that, at least on-node, the integration of MPI and GPU backends is not optimal. The usage of MPI is a necessary technology for high efficiency. Yet, the challenge of exponential memory will limit the state vector simulation below 50 qubits.

Two main approaches could address these limitations. The first strategy is to adopt hybrid approaches, combining, for instance, state vector and Feynman path simulators [23] or approximation techniques, such as the tunable Schmidt-decomposition rounding-parameter (SDRP) approximation [35], trading memory usage for increased computation or simulation fidelity. Because these hybrid and approximated approaches use the state vector simulator technology, they are capable of exploiting GPU acceleration, as shown in this work. A second strategy is exploiting the state vector's sparsity and using compression. We note that the Qiskit Aer and many other state-of-the-art simulators use dense state vectors. Important considerations for future work should be

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.

sparse representations, for instance, in bitwise-representation quantum computer simulators [36], and usage of sparse linear algebra libraries for GPUs.

ACKNOWLEDGMENT

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Sweden, Finland, Germany, Greece, France, Slovenia, Spain, and the Czech Republic under grant agreement No 101093261.

REFERENCES

[1] M. I. Andersson, N. A. Murugan, A. Podobas, and S. Markidis, "Breaking down the parallel performance of gromacs, a high-performance molecular dynamics software," in *International Conference on Parallel Processing and Applied Mathematics*. Springer, 2022, pp. 333–345.

[2] G. Schieffer and I. Peng, "Accelerating drug discovery in autodock-gpu with tensor cores," in *Euro-Par 2023: Parallel Processing: 29th International Conference on Parallel and Distributed Computing, Proceedings*. Springer, 2023.

[3] M. Karp, D. Massaro, N. Jansson, A. Hart, J. Wahlgren, P. Schlatter, and S. Markidis, "Large-scale direct numerical simulations of turbulence using gpus and modern fortran," *arXiv preprint arXiv:2207.07098*, 2022.

[4] S. W. Chien, J. Nylund, G. Bengtsson, I. B. Peng, A. Podobas, and S. Markidis, "sputnipic: an implicit particle-in-cell code for multi-gpu systems," in *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 2020, pp. 149–156.

[5] O. Fuhrer, T. Chadha, T. Hoefler, G. Kwasniewski, X. Lapillonne, D. Leutwyler, D. Lüthi, C. Osuna, C. Schär, T. C. Schulthess *et al.*, "Near-global climate simulation at 1 km resolution: establishing a performance baseline on 4888 gpus with cosmo 5.0," *Geoscientific Model Development*, vol. 11, no. 4, pp. 1665–1681, 2018.

[6] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[7] P. Gokhale, A. Javadi-Abhari, N. Earnest, Y. Shi, and F. T. Chong, "Optimized quantum compilation for near-term algorithms with openpulse," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 186–200.

[8] H. Silvério, S. Grijalva, C. Dalyac, L. Leclerc, P. J. Karalekas, N. Shammah, M. Beji, L.-P. Henry, and L. Henriet, "Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays," *Quantum*, vol. 6, p. 629, 2022.

[9] I. L. Markov and Y. Shi, "Simulating quantum computation by contracting tensor networks," *SIAM Journal on Computing*, vol. 38, no. 3, pp. 963–981, 2008.

[10] I. L. Markov, A. Fatima, S. V. Isakov, and S. Boixo, "Quantum supremacy is both closer and farther than it appears," *arXiv preprint arXiv:1807.10749*, 2018.

[11] X.-C. Wu, S. Di, E. M. Dasgupta, F. Cappello, H. Finkel, Y. Alexeev, and F. T. Chong, "Full-state quantum circuit simulation by using data compression," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–24.

[12] Qiskit Community, "Qiskit: An open-source framework for quantum computing," Mar. 2017. [Online]. Available: https://github.com/Qiskit/qiskit

[13] M. A. Nielsen and I. Chuang, "Quantum computation and quantum information," 2002.

[14] T. Häner and D. S. Steiger, "5 petabyte simulation of a 45-qubit quantum circuit," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017, pp. 1–10.

[15] J. Doi, H. Takahashi, R. Raymond, T. Imamichi, and H. Horii, "Quantum computing simulator on a heterogenous hpc system," in *Proceedings of the 16th ACM International Conference on Computing Frontiers*, ser. CF '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 85–93.

[16] J. Doi and H. Horii, "Cache blocking technique to large scale quantum computing simulation on supercomputers," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2020, pp. 212–222.

[17] cuQuantum Development Team, "cuquantum," Apr. 2023. [Online]. Available: https://doi.org/10.5281/zenodo.7806810

[18] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," *arXiv preprint arXiv:1707.03429*, 2017.

[19] N. Bell and J. Hoberock, "Thrust: A productivity-oriented library for cuda," in *GPU computing gems Jade edition*. Elsevier, 2012, pp. 359–371.

[20] A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation, and J. M. Gambetta, "Validating quantum computers using randomized model circuits," *Physical Review A*, vol. 100, no. 3, p. 032328, 2019.

[21] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994, pp. 124–134.

[22] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.

[23] S. V. Isakov, D. Kafri, O. Martin, C. V. Heidweiller, W. Mruczkiewicz, M. P. Harrigan, N. C. Rubin, R. Thomson, M. Broughton, K. Kissell *et al.*, "Simulations of quantum circuits with approximate noise using qsim and cirq," *arXiv preprint arXiv:2111.02396*, 2021.

[24] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1996, pp. 212–219.

[25] D. M. Greenberger, M. A. Horne, and A. Zeilinger, "Going beyond bell's theorem," *Bell's theorem, quantum theory and conceptions of the universe*, pp. 69–72, 1989.

[26] A. Montanaro, "Quantum algorithms: an overview," *npj Quantum Information*, vol. 2, no. 1, pp. 1–8, 2016.

[27] Nvidia, "Nvidia nsight systems," 2023. [Online]. Available: https://developer.nvidia.com/nsight-systems

[28] ——, "Nvidia nsight compute," 2023. [Online]. Available: https://developer.nvidia.com/nsight-compute

[29] K. De Raedt, K. Michielsen, H. De Raedt, B. Trieu, G. Arnold, M. Richter, T. Lippert, H. Watanabe, and N. Ito, "Massively parallel quantum computer simulator," *Computer Physics Communications*, vol. 176, no. 2, pp. 121–136, 2007.

[30] H. De Raedt, F. Jin, D. Willsch, M. Willsch, N. Yoshioka, N. Ito, S. Yuan, and K. Michielsen, "Massively parallel quantum computer simulator, eleven years later," *Computer Physics Communications*, vol. 237, pp. 47–61, 2019.

[31] J. Doi *et al.*, "Efficient techniques to gpu accelerations of multi-shot quantum computing simulations," *arXiv preprint arXiv:2308.03399*, 2023.

[32] S. Imamura, M. Yamazaki, T. Honda, A. Kasagi, A. Tabuchi, H. Nakao, N. Fukumoto, and K. Nakashima, "mpiqulacs: A distributed quantum computer simulator for a64fx-based cluster systems," 2022.

[33] S. Markidis, S. W. Der Chien, E. Laure, I. B. Peng, and J. S. Vetter, "Nvidia tensor core programmability, performance & precision," in *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, 2018, pp. 522–531.

[34] J. Wahlgren, M. Gokhale, and I. B. Peng, "Evaluating emerging CXL-enabled memory pooling for HPC systems," in *2022 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. IEEE, 2022, pp. 11–20.

[35] D. Strano *et al.*, "Exact and approximate simulation of large quantum circuits on a single gpu," *arXiv preprint arXiv:2304.14969*, 2023.

[36] E. C. R. da Rosa and B. G. Taketani, "Qsystem: bitwise representation for quantum circuit simulations," *arXiv preprint arXiv:2004.03560*, 2020.

Authorized licensed use limited to: Mackenzie University (Universidade Presbiteriana Mackenzie). Downloaded on April 17,2026 at 23:08:11 UTC from IEEE Xplore. Restrictions apply.